Bitwise operator -

- Right Shift

32 >> 1

↓

10000 → 01000 = 16

- Left Shift
- variable varies in circular range of integer & other data types.

-32768 to 32767

→ if range increases to maximum, then it resets to minimum & vice versa.

- :set nu → set line no.
- int a → dec^n + def^n
  int b = 10; → dec^n + def^n + Init
  declaration → no memory no declaration

	Name	Memory	Address	Value
dec	✓	X	X	X
def^n	✓	✓	✓	X
Initia	✓	✓	✓	✓

- extern int → only dec.
- :: → scope resolution operator
- Use :: to operator to use ex global extent value.

⊙ Decision Control Structure

→ if
→ if else
→ switch case

Data type	Size	Range
int	4 bytes	-2,147,483,648 to 2,147,483,647
float	4 bytes	
long	8 bytes	
double	8 bytes	
char	1 byte	-128 to 127
short	2 bytes	-32768 to 32767
long int	8 bytes	
unsigned int	4 bytes	0 to 4,294,967,295
unsigned long	8 bytes	
long double	16 bytes	
void	1 bytes	

⊙ exit( ) → library function

→ it terminates program execution
→ exit(0) → graceful exit
→ exit(1) → erroneous exit

⊙ Loops → used to perform iterative task.

Types of loops -

→ Counter controlled loops
* → Sentinel controlled loops → (Sentimental Decision by user) Do you want to continue or not
→ Entry controlled loops
* → Exit controlled loops

⊙ Difference b/w for & while

for loop → only concentrate on purpose of loop

while loop →

⊙ for (; ;) ; ⟶ empty statement.

⊙ per = float(a/b); or per = float(a)/b;

↓ It will convert a to float not $\left(\frac{a}{b}\right)$.

per = a / float(b);

↳ It will convert b to float not $\left(\frac{a}{b}\right)$;

same o/p

per = 41

⊙ if (per > 40, per < 40) ⟶ false

→ if if statement is seprated by comma operators, then only last condition will work.

per = 41

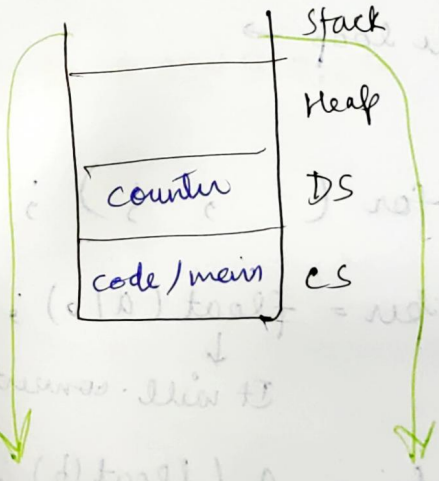
- if (per > 40 , per = 20, per < 40) → true.
- use capital letters & underscore for constants only.

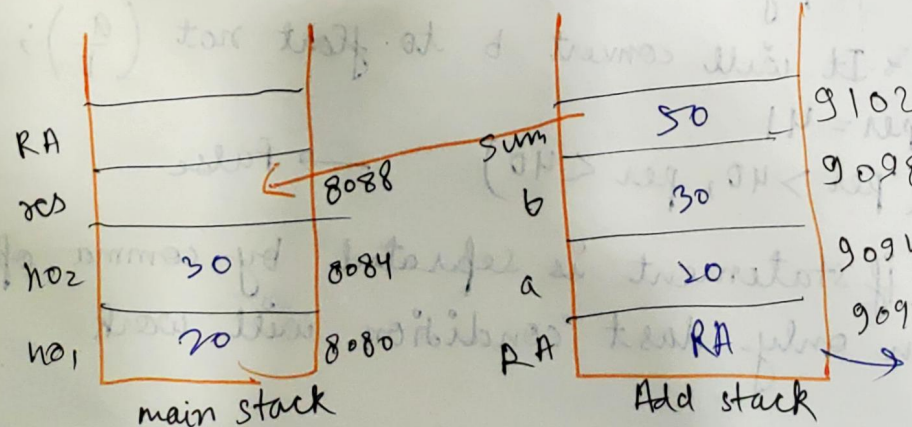
Function -

Recieve → Process → Produce → Return

- main() is user-defined function, because it is define by user.

```
int add (int a, int b){
    static int counter = 0;
    int sum=0;
    sum = a+b;
    counter++;
    return sum;
}
int main ( ){
    int no1, no2;
    cin >> no1;
    cin >> no2;
    int res= Add (NO1, No 2);
}
```





- Transfer data from one stack to another known as pass by value.
  add stack ⟶ main stack
- Static variable & global variable works throughout the program because it falls in data segment.
- Local variable falls in stack segment that's why they work throughout the function.

Buffer -

- Buffer exists for some time.
- Buffer stores some amount of data. We can add small small amount of data till limit

1> input parameter
2> return address
3> frame pointer
4> Local variable
5> exception handling
6> Buffer
7> callee save registers

Header -
Sepration of specification & implementation.

Library -

Implementation.

Factorial.h

```
int fact(int);
```

→ Header

Factorial.cpp

```
#include "Factorial.h"
int fact(int a){
    int res = 1
    while(a>0)
    {
     res*=a;
     a--;
    }
}
```

↓ Library

FactorialTester.cpp

```
#include "FactFaniale.h"
#include <iostream>
using namespace std;
int main()
{
   int n;
  Factori
   cin>>n;
   cout<< fact(n);
   return 0;
}
```

→ header used in this file

ld → linker error

⊙ write dependent file in last

⊙ g++ Factorial.cpp FactorialTester.cpp
dependent

It will compile Factorial.cpp first.

⊙ Factorial.h.gch → gnu compiled Header
compiled header
.pch → pre compiled header

⊙ remove gch file always, otherwise it will use not modified header.

rm *.gch

Array name is hidden constant pointer.

Array - (Constant Pointer)

```
char name[50];
name = "Kriti";
cout << "\n" << name;
```

const pointer → error

we can not assigned Kriti to name because it is read only.

- const on LHS of '=' (Assignment operator) → can not change

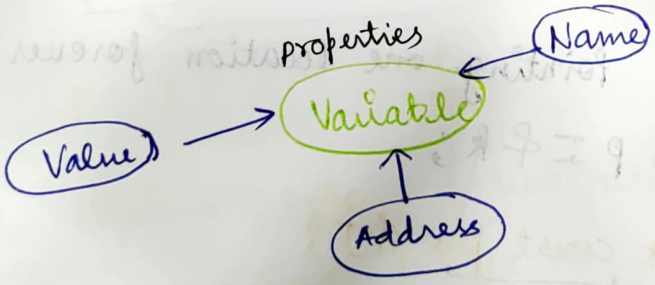
eg.

L value required - Modifiable variable required

- Intialization of constant is allowed ✓ but assignment of constant is not allowed. ✗

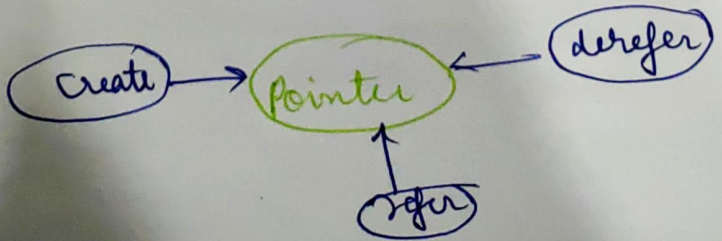
(K with datatype) int K = 30 → intialization

(K without datatype) K = 60 → assignment



Pointers:

```
int *p;          // uninticalised pointer
cout << &p;
Cout << *p;
```



Uses -

- Access memory without name
- access data in different stack frame
- pass by refrence

```
int * p = NULL;      → C
int * p = nullptr;   → C++
```
} Null pointer

Fancy Pointer - Overwritable

we can assigned it to any memory location.

```
int K = 100;
int J = 50;
int * p = nullptr;
p = &K;   → pointing to K
p = &J;   → pointing to J
```

Constant Pointer - Pointing one location forever.

```
int * const p = &k;
```

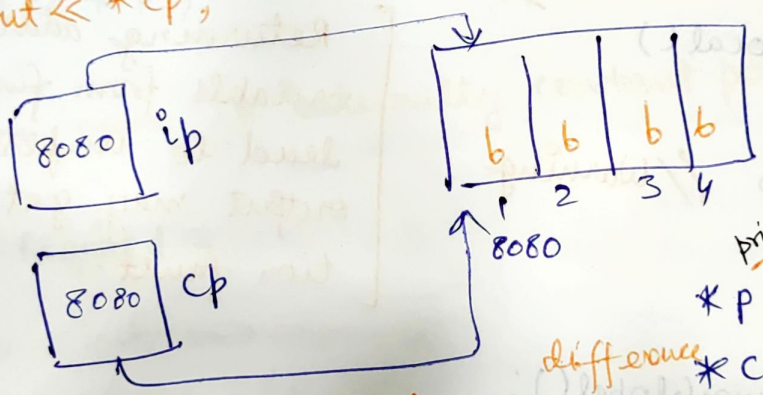
```
const int * const p = &j;
```
const datatype — const pointer

error
- p++; → read only variable
- (*p)++; → read only location
- J++; → read only variable

Pointer Conversion :

- Any type of pointer can be type casted to any other type with proper type casting.
- size of any type of pointer is same as which is equal to size of unsigned integer

```
int k = 65;
int *ip = &k;
char * cp = (char*) ip;      // Type casting
cout << *cp;
```



ip = cp

but

difference

*p — read one integer

*cp — read one char

prints 65

prints A

- pointer size → 8 byte
- pointer is unsigned integer. / unsigned int

Void pointer -

```
int * a;
void * p = a;      // No casting required for generic pointer.
```

Pass by Address / refrence / pointer -

- transfer of values b/w different stack frames & we can also access values of different stack frames.

Return address -

```
int k = 100;
int * ChangeGlobal()
 {
   return &k;
 }
int * ChangeLocal()
 {
   int i = 10;
   return &i;  //Warning..
 }
int main(){
  int *p1 = ChangeGlobal();
  cout << *p1;          // print 100
  int *p2 = changeLocal();
  cout << *p2;          // Unpredictable output
  return 0;
}
```

→ Throughout the program

This stack frame distroyed after returning

Returning address of local variable from function may lead to unpredictable output may get segmentation fault.

- Pointers & constant
  - Pointer to constant

- Constants Pointer to constant value pointed by constant pointer to constant can not be changed also pointer can not be changed

```
#include <iostream>
{
const int k = 20;
const int * const p = &k;
cout << * p << endl;
 p = NULL;
 p = p+1;
 * p = 100
 cout << * p;
}
```

Not possible

- Function name is internally constant pointer

R-Value Required -

```
# include <iostream>
 using namespace std;
void printData (int i)      // return void
   cout<<i;
int main()
 { int i = printData(5);    // expecting int
 }
```

- Statement which not allowed to give value should not RHS.
- Read only will not allowed on LHS.

- Scaling of data can be possible by making it constant.
- Character have termination character '\0', so no need to pass size to function.

Command to compile with specific version – g++ -std=c++98 Array.cpp

Memory Allocation –

C

- malloc function used to allocate
- does not understand a datatype.
- returns generic pointer (void *) / Raw pointer
- e.g. allocate for 10 integer
  - malloc (10 * sizeof(int));
  - calloc (10, sizeof(int));
- deallocation
  - free (p); → int or 10*int

C++

- new operator used to allocate.
- understand a datatype.
- returns specific pointer
- → int *p = new int[10];

Write operation ← *p = 100
int z = *p → Read operation

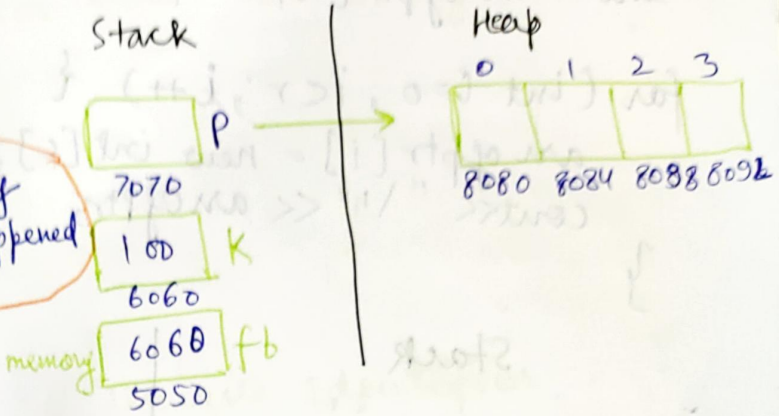
- deallocation
  - delete p; → int
  - delete []p; → 10*int

- Memory leak – Allocated memory which is not deallocated & not in use, Then memory leak arises.

⊙ Dynamic Allocation -

```
int * p = new int[4]; // Dynamic allocation
int k = 100;
int * fp = &k;
fp = p;
p = fp;
```

p = fp; → memory leak happened



p created on stack & memory referenced by p created on heap

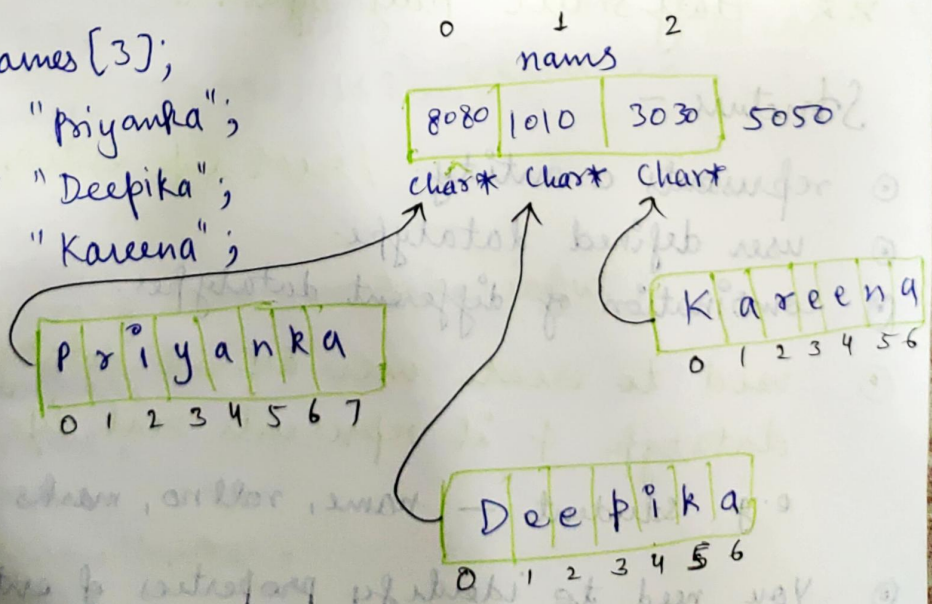
We can not access allocated memory again

⊙ if we make p constant, we can not intiallize it again.

```
int * const p = new int[4];
```

⊙ Array of pointers

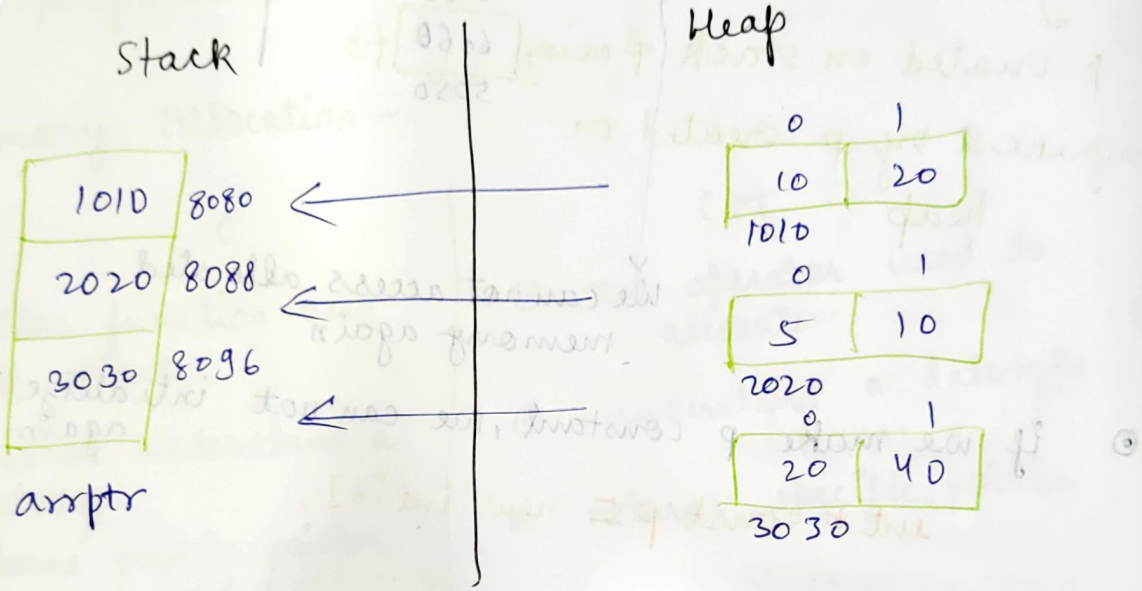
```
char * names[3];
names[0] = "Priyanka";
names[1] = "Deepika";
names[2] = "Kareena";
```



Array of pointer dynamic –

```
int * arrofptr[3], r=3, c=2        → static allocation

for (int i=0 ; i<r ; i++) {
    arrofptr[i] = new int[c];      → dynamic allocation
    cout<< "\n" << arrofptr;
}
```



★★ Half static Half Dyanmic

Structure –

- represents a entity / real life entity
- user defined datatype.
- combination of different datatypes.
- used to create user defined datatype from existing datatype & it represents real life entities.

  e.g. student – name, roll no, marks etc.

- You need to identify properties of entity. (properties ↓ members)
- possible datatype for ~~entity~~ properties
- possible default value for ~~entity~~ properties

- Person struct - ***
- Need calculation ⟶ use numeric data type (int, float, long)
- No need of calculation ⟶ use string, char etc

```
# include <iostream>
using namespace std;
{
   Struct Person
   {
    char aadhar [12];   // Unique Identifier
    char name [30];
    int age;
    char phoneNumber [10];
   };
int main() {
    Person p1;
    p1.aadhar = "525291192923";   } error
    p1.name = "Harshit";          } we can not assign value to constant
    p1.age = 23;
    p1.phoneNumber = "7233071267"; } error
    return 0;
}
```

Instead of assigning data, we take input;

```
cin >> p1.aadhar;
cin >> p1.name;
cin >> p1.age;
cin >> p1.phoneNumber;
```

Pointers in structure

```
int main() {
    Person P1;
    Person * pp1 = &P1;        // Pointer variable
    cin >> P1.aadhar;
    cin >> pp1 -> aadhar;      } Both are same
}
```

Protect

```
void Display ( const Person * const pp)
```

Both should be constant by which no one can change your data.

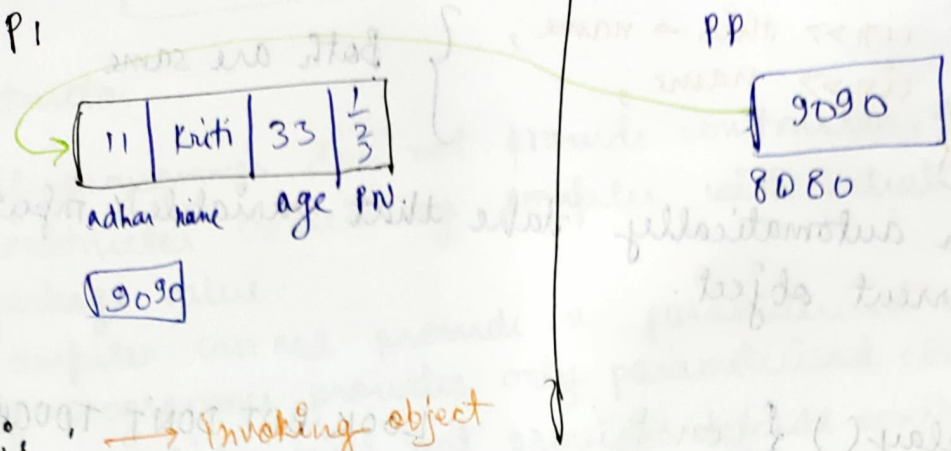
```
void Accept ( Person * const pp)
```

Should not be constant ↓ we need to modify it.

Should be constant ↓

- If the function is not globally defined we need to access it using structure variable.

Main Global fuction | Accept member function



this : ⟶ invoking object

- "this" is a pointer to object/variable of class/structure.
- this pointer holds address of invoking object/variable
- 'this' pointer is secretly created by compiler.
- this is a constant point to invoker
- * this = value at this will be invoking object always.

this is constant pointer to invoking object. **

Person * const this = &...

Default

```
void Accept ( Person * const PP)
{
    cout<< PP;
    cout<< this;
}
```

Both will give same value.

- No need to send parameter in the function, we can work with this pointer.

* ** this only applicable/available inside Member function only

⊙ ~~CIN~~ Void Accept ( )

```
{
    cin >> this -> name;
    cin >> name;
}
```

} Both are same

⊙ compiler automatically take that variable's input in current object.

```
void Display ( ) { const
{
    cout << this -> aadhar ;
    cout << this -> name ;
}
```

LOOK BUT DON'T TOUCH

⊙ constant member function makes invoking object / variable oread only / write protected / constant.

Class & Object

Class is a programming construct which defines data member & member functions of objects & control the access of data.

Visualization of object → class

⊙ class is like a skeleton, blue print.
⊙ Objects are with different states & behaviours.

State – Attributes
Behaviours – Method

Constructor –

- If programmer does not provide constructor, then default constructor created by compiler with intiallised with garbage value.
- compiler can not provide a parameterised constructor, if programmer provides only parameterised constructor then compiler will not provide default constructor.
- If class provide constructor, compiler will not provide constructor.

Person :: Person ( ) —— No matching function error
↓ ↓
type name person function name

solution |

** when you create parameterised constructor, make default constructor also.

```
int i=10;
i=i;
i=i=i=i;
```
> Self Assignment

```
Void display()
{
  cout << this -> aadhar;
  cout << (*this).aadhar;
}
```
} Both are same

↓ invoking object

- Destructor only required for important & valuable or private data.

```
#include <iostream>
using namespace std;
class complex
{
    private:
    int real;
    int img;

    public:
    void Accept()
    { cout<<"Enter real & img data";
      cin>>real>>img;
    }
    void display() const
    {
      cout<<"\n Complex["<<real<<"+i"<<img<<"]"<<endl;
    }
    complex(): real(0),img(0) // Default constructor with intialization list.
    {
    }

complex()
{
    real=0;          // Assignment
    img=0;
    cout<<"\n Complex:: Complex"<<endl;
}
```

Before invoking constructor

Pure intialization

after invoking Constructor

```
class complex
{
    const int c;

    complex() : c(10)          // working
    {
    }
    complex()
    {
      c=10;                    // Error
    }
```

→ intialised before constructor excules

→ intialised after constructor execute

⊙ Constant data members must be intiallised in intiallization list of constructor. (not in konstructor body)

** static variable shared among all the objects.

Getter → Accessor
Setter → Mutator

In object - Instance-member - object member
Not in object - non-instance member - class member

⊙ Things which are not object member should not be intiallised in constructor.

⊙ class member such as static member can not be intiallised in constructor.

⊙ ** Static data members must be intiallised outside the class with scope resolution.

- Static is used in only in case of declaration not intiallization. (only once)

```
// Static DMInit
int complex :: count = 0;   / int complex :: count(0);

int main ()
{
    int a1;      → like default constructor
    int a(10);   → like parameterised constructor
    int b(a);    → like copy constructer
    int c=a;       dec^n + def^n + int^n with existing object
    int a=10;
```

Implicit constructor conversion

- Static member functions are called using class name & scope resolution operator.

e.g.

```
class complex {
    static int GetCount () {
    }
};
complex :: getCount ();
```

- Because static member fun^n called using classname Hence these function will never receive this pointer

Static	Non static
class member	object member
do not recieve this	receive this
only static dM	Both static & non-static DM

Object creation ways -

1. Complex c1; → object creation & called default constructor
2. Complex *cp = new Complex; dynamic object creation & called to default constructor.
   delete cp;
3. Complex c2(10,10); → object creation & called parameterized constructor
4. Complex *cp2 = new Complex(10,10); →
   delete cp2;
5. Complex c3(c1); // Copy constructor
6. Complex c4 = c1;
7. Complex c5 = Complex(10,10);

```
// Normal array
Complex cars[10];
car[0]
// Dyamic array
Complex *cptr = new Complex[10];
// cptr[0];
delete []cptr;
```

// Add fuction -

```
Complex Add(const Complex temp) const → make data mem constant
{
    complex result;
    result.real = this->real + temp.real;
    result.img = this->img + temp.img;
    return result;
}
```

```
Complex Add (const Complex temp) const {
    return Complex(this->real+temp.real, this->img+temp.img);
}
```

Complex.h

```
Class Complex {

    private:
     int img;
     int real;
    public:
    Complex() {
    }
    Complex( int real, int img) {
    }
    void Display() const
    {
    }
    void A::ComplexAdd (Complex ())
    {
    }
}
```

- If the function differ by no. of argument, order of argument, type of argument.
- Same function name with different argument list.
- Function overloading.

```
int add (int a, int b) { }
int add (float a, float b) { }
int add (int a, float b, float c) { }
```

} overloaded add function

- return type ignored in function overloading

↓

Because constructor do not have return type & Hence it ignore return type. After that it applied to function. (function overloading).

- Compiler dependent process → Name mangling/ Name declaration

How compiler distinguish?

```
Add@i@i
Add@f@f
Add@i@f@f
```

→ Add@2@int@int

or

Different compiler have different naming methods.

Operator Overloading-

Operator overloading is in what implement/ define/ train operator for user defined data type.

$C_3 = C_1 + C_2$

⇓

$C_3 = C_1.operator+(C_2);$

Complex operator+ (complex c) {
}

Properties

- All operator can not overloaded

e.g. ::, ., →

- Some operator load as memberfunction & some of the operator can be overloaded as global function.
- You can only overload existing operator.
- Associativity & prefidence of the operator can not be change using operator overloading.
- Insertion operator, Extraction operator

Memory Leak -

The memory allocated dynamically but not delited / freed is called as memory leak.

std :: bad_alloc - Exception which will be thrown when allocation of memory is not passible using new. (bad allocation)

- It will created & destroy for a long time when you use delete keyword.

```
Void function_Memory_leak() {
    int * p = new int [5000000000];
    cout << "Allocated";
    delete []p;
}
```

```
int main(){
    for(int i=0; i<8000000000000;i++){
            function_memoryleak();
    }
    int *p= new int;
    *p=10;
    cout<<*p;
    delete p;
    return 0;
}
```

Dangling Pointer -

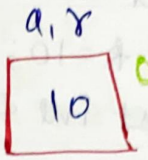
```
# include <iostream>
using namespace std;
int main()
{
    int *p1 = new int;
    int *p2= p1;
    *p2 = 100;
    cout<<*p1<<endl;
    delete p1;
    cout<<*p2<<endl; // segmentation fault // p2 is dangling pointer
    delete p2;
    return 0;
} // p2 is trying to access memory deleted by p1.
```

References -

- Reference is alias for variable.
- How to declare & define reference

```
int a = 10;
int &r = a;
```

Here r is reference to a (alias)



outside ✓
inside X

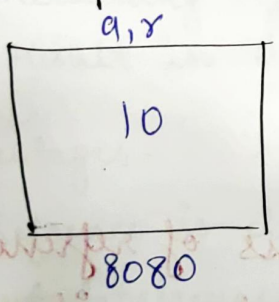
- Reference has to be intiallised where it is declared but can not be intialized with NULL.

```
int &r;
```
X

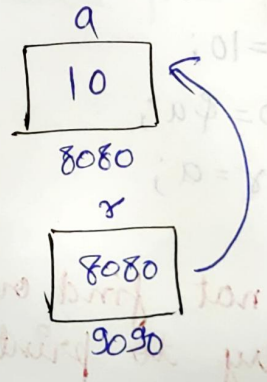
- Reference is internally a constant pointer hence needs to be intiallised when it is declared.
- It is internally a constant pointer.

reference (pointer
constant pointer

Conceptual



Internal



- No use of * to use reference
- References are auto dereferenced
- Change in reference will change original variable value.

```
int a = 10;
int &r = a;
r = r+1;
cout << r << a;   // 11 11
```

- Refrence to const is also possible. It is also known as const refrence

```
const int &r=12;
int k=10;
const int &r1=k;
```

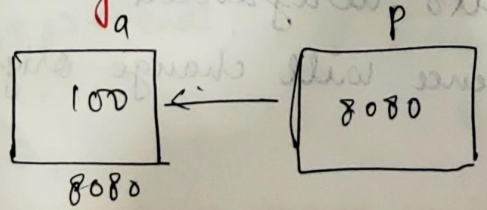
- We can not access dynamically allocated memory using refrence.
- We can not change value of variable / constant using refrence if refrence is const refrence.

```
const int &r=12;
r=r+1; // will generate error
```

Pointer & refrence declaration difference

```
int a=10;
int *p=&a;
int &r=a;
```

- We can not find out original address of refrence if we try to print address of the refrence it will print address of a original variable to whom it is referring.



$*(\&a) = *(8080) = 100 = a$

$\&(*p) = \&(100) = 8080 = p$

☆☆ `*(&a) = a`
`&(*p) = p`

जहाँ जहाँ & वहाँ वहाँ *

⊙ `int &r = a;`


```
r = r+1
  ⇓
*r = (*r+1);
```
} Automatic

(&r) ⟹ reference are auto dereferenced

`&r ⇒ (&*r) = (&a)`

⊙ Use conceptual reference, internal is for concept only.

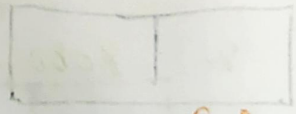
Passing reference to function (pass by reference)

⊙ Changes made in a reference reflects in a original variable & vice versa.

⊙ Returning reference of local variable from function will result in dangling pointer

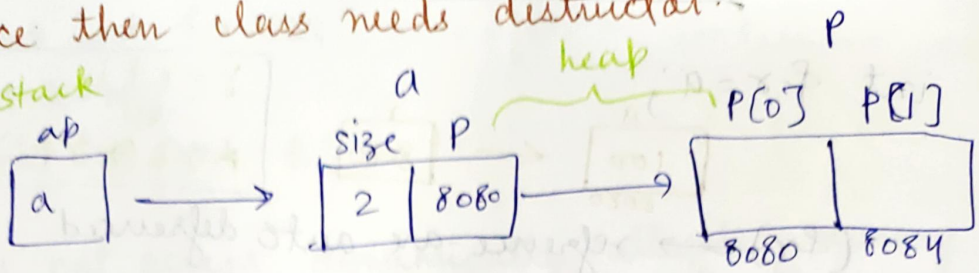
Advantages -

1. Reference is clean
2. autodereferenced
3. Minimize use of pointer



☆☆ Size of empty class is 1 byte because minimum allocatable memory in the system is 1 byte.

- If data members is allocated dynamically and it holds address of a heap memory / any resource then class needs destructor.

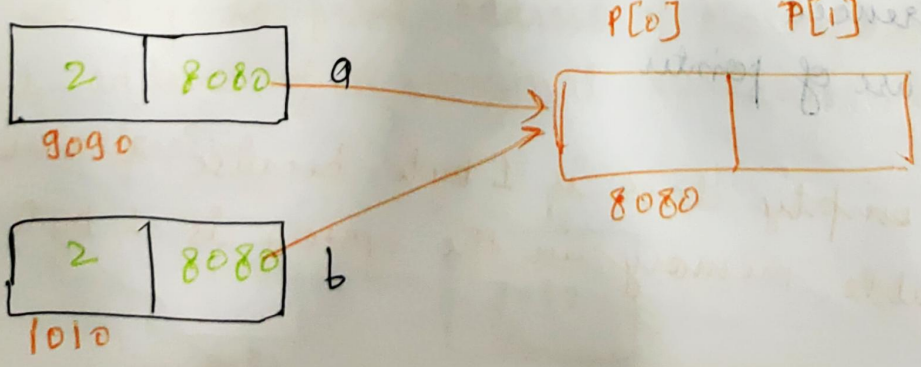


→ when we delete ap automatically a & P deleted

Copy constructor – creating new object from existing object

```
Array b(a);   // copy ctor
b = a;        // not copy ctor ( b already exist )
```

- If programmer does not define copy constructor, then compiler will provide default copy constructor which performs SWALLOW COPY.



- Swallow copy is not good for program.

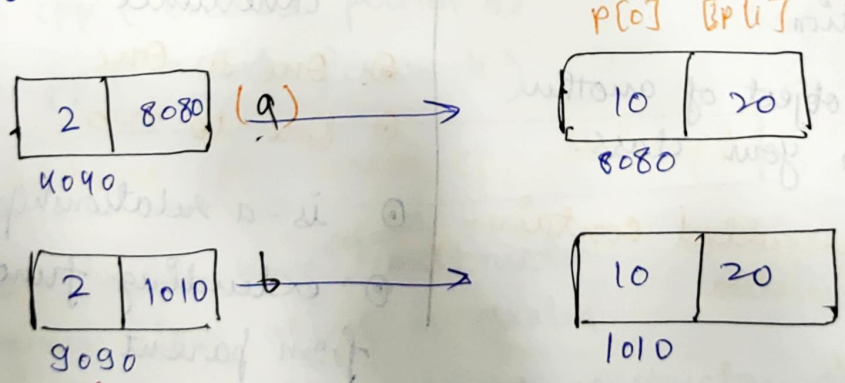
```
main() {
    Array a(2);
    a.SetIndex(0,10);
    a.SetAtIndex(1,20);
    a.display();
    {
     Array b(a);
     b.display();
    }
    a.display();
    return 0;
}
```

// double free detected in tcache 2
// Aborted (core dumped)

→ going out of scope

Dip Copy -

1 → Copy the size
2 → Allocate memory of that size
3 → copy content of the original object.




```
Array (const Array& x) { // Array b(a); x ---> a, this ---> b
    this->size = x.size;                    // 1st step
    this->p = new int[this->size];          // second step
    for(int i=0; i<this->size; i++)         // 3rd step
    {   this->p[i] = x.p[i];
    }
}
```

⊙ Function call can be used on L.H.S. of assignment operator if it returns reference of non-locals.

```
int main(){
    Array a(2);
    a.At(0) = 10;        →  a[0] = 10 → a.operator[](0) = 10;
    a.At(1) = 20;
    cout << a.At(0);     // internally ref of p[0]
    cout << a.At(1);     // internally ref of p[1]
}
```

Encapsulation Vs Data Hiding

Encapsulation: More about linking data members with functions.

Data Hiding: More about hiding data.

Association Vs Inheritance

Association

- ⊙ If using same functionalities as it is, called association
- ⊙ Creating a object of another class into your class.
- ⊙ also ~~some times~~ called contain-ment
- ⊙ One to One
- ⊙ One to many

1. Loose coupling
2. Tight coupling

- ⊙ has-a relationship

Inheritance

- ⊙ If using with little modification is called inheritance.
- ⊙ One to One
- ⊙ One to two - - -
- ⊙ is-a relationship.
- ⊙ extending functionalities from parent

OOP Principle — Encapsulation

coupling ↓, cohesion ↑

High cohesion ↑ — You are not much dependent on another.

Association — Both side has ——(no dirn)→ Driver has car, car has driver
Loose coupling

Aggregation — One side has ⟶(direction) Library has books, book has library ✗
Moderate coupling

Composition — Part of relationship → Person has heart (You can separate) (High dependency)
Highest coupling

Steps

1. Create a file Address.h (declare address class)
2. Address.cpp (include Address.h)
3. Person.h (declare Person class & include Address.h)
4. Person.cpp (include person.h)
5. Main.cpp (include Person.h)

```
g++ Address.cpp Person.cpp Main.cpp
```

- Both classes have default constructor.
- Container class object is created.

Container — Person
Contained — Address

Case 1:
- Automatically call default constructor of contained class.

Case-2

Container has default constructor but contained has only parameterised constructor.

- When you call a constructor of another class, call, then we need to call parameterized constructor of contained class from constructor of initialization list of the container.

Case-3 - Both class do not have default constructor. only parameterised ctor.

- default ctor called from default ctor of paramet. erised ctor called from parameterised ctor

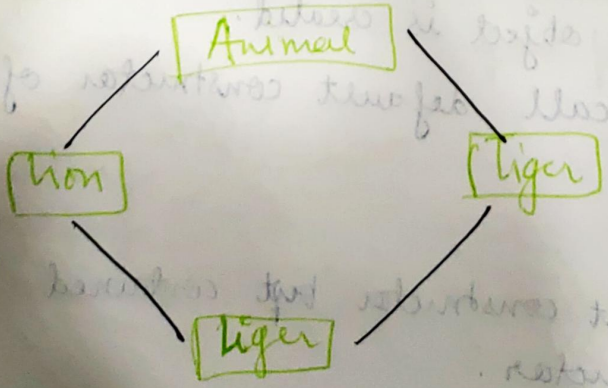
Best practice

Prefer association over Inheritance.

Inheritance -

- Inheritance is capibility of one class to acquire properties & behaviours of another class.
- Runtime Polymorphism can not be done without inheritance.
- Extends property of parent
  - Inherit & modify

Diamond Problem, Hybrid Inheritance -



topline → sale
expenses
bottomline → profit

topline - expenses = bottomline

getline (cin, name) → global
cin.getline(name) → local

```
class CktPlayer : public Player
```
syntax

Deligating - calling Parent class function in child class.

Ctor → B to D.
Dtor → D to B.

calling Base class default ctor from derived class -

```
CktPlayer :: CktPlayer (string name, int age, int runs) :
                Player :: Player (name, age), runs(runs)
```

↓ constructor chaining

Case 1: Derived class default ctor → calls → base class default ctor

Case 2: Derived class para ctor → calls → base class para ctor

Case 3: If default ctor not present in base class

cons.

Upcasting -

Parent class pointer / refrence can point to child class object provided that inheritance is public.

```
Player *pp = nullptr;
pp = &c;          // upcasting

Player &pr = c;   // upcasting
```

Compile time Binding -

Compiler — Blind bind

- If caller type have that function in class. It will bind.

caller type → Player, caller → *pp

```
Player *pp = nullptr;
pp = &c;  → pp→display();
```

function available

- Hold until Runtime

↓

We can achieve this using virtual keyword → .h

- Just write virtual in one file only no need to write in both .h & .cpp file.

- Runtime binding

Virtual function present	Virtual function not present
Run time binding	Compile time binding

Three important things for

1- Runtime Polymorphism

1. Upcasting
2. virtual function — Hold until runtime & resolve the call at runtime by checking the object type.
3. overriding.

If one class defined multiple times -

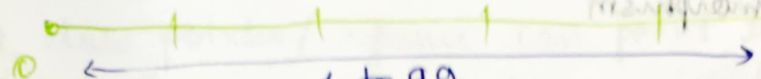
```
#ifdef ____PLAYER_H__
#define ____PLAYER_H__
#include <string.h>
using namespace std;
class Player
{
    private:
        string name;
    public:
        void display();
        void accept();
};
#endif
```

Header guard

Pure Virtual functions -

- If a virtual function is mentioned with equal to 0 is known as pure virtual function i.e. abstract function. (PVF)
- If a class have atleast one pure virtual function then it is called as a abstract class.
- If class has all the functions as PVF, then it is called as interface.

Abstraction line



Concrete class (Fully implemented)

1 to 99 ↓ Abstract Class

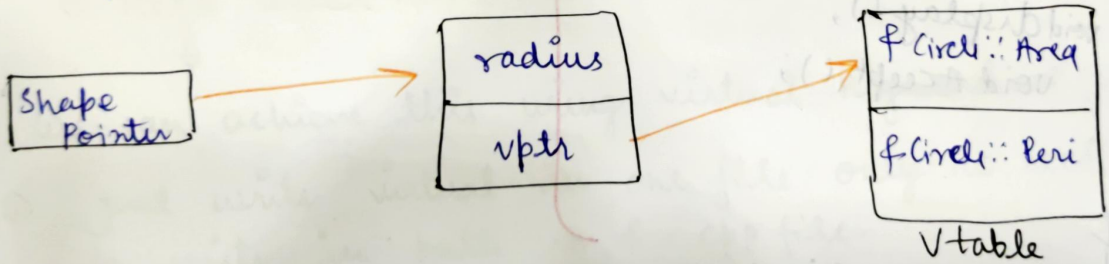
Interface (Pure virtual functions)

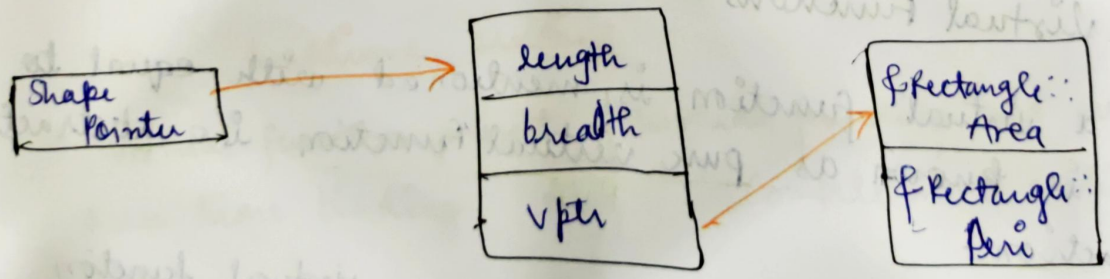
- Object of interface / abstract class can not be created.
- We can create objects of only concrete class.

Virtual table & Virtual Pointer -

When there is one virtual function present in class will create virtual table.

- This class have one more data member secretly called vptr.

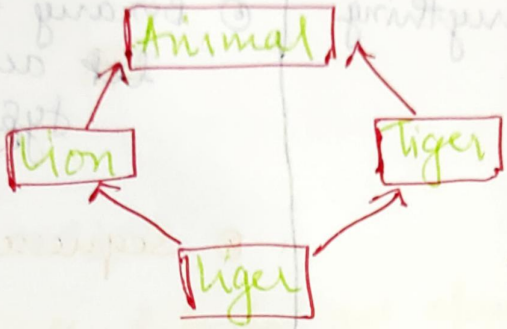




- Initialization of vptr happens in constructor -

- Constructor is the only thing which we can not make virtual, because vptr intialize in it.

Diamond Problem – (Interview)



- Where, there multiple inheritance from sibling, then diamond problem arise.
- If Lion inherit weight from Animal & tiger inherit weight from Animal then liger inherit two times weight. This is the problem.

Solution 1.

- Inherit it virtually.

```
class Lion : virtual class Animal
```

Solution 2. call that function with particular class name

```
Liger lg;
lg.Lion :: setWeight();
```

file handling -

Text File	Binary file
e.g. 123456 6 char / 6 byte	e.g. 123456 int / 4 byte
⊙ Text file take everything as characters	⊙ Binary file take differently ~~text~~ according to data type.
⊙ Sequence of char	⊙ Sequence of binary

File opening Modes — Meaning

→ ios::out, ios::in, ios::app, ios::ate, ios::trunc, ios::binary

bof - begin of file
eof - end of file

I/O stream Library & Standard Stream Objects -

⊙ <iostream> → basic I/O (istream, ostream, iostream)

⊙ <fstream> → file handling (ifstream, ofstream, fstream)

Standard stream objects -

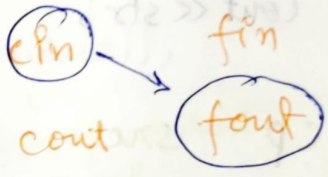
cin - istream class - input
cout - ostream class - output
cerr - ostream class - standard error output
clog - ostream class - also to standard error, buffered

```
cout.put('A');
cin.get();
cin.getline(buffer, size);
cin.eof();
```

Resource leak - If opened a file & forget to close.

getline / file → input from console & write to file

```
int main( )
{
   ofstream fout;
   fout.open("abc.txt");
   if(!fout)        // file is not open
   { cout<<"Unable to open file";
     return 1;
   }
   char ch;
   do
   {
   string str;
   cout<<"\n Enter string"<<endl;
   getline(cin,str);
   fout<<str;
   cout<<"\n Do u want to continue (y or n)"<<endl;
   cin>>ch;
   }while(ch!='n');
    fout.close();
  return 0;
}
```



Harshit

y

cin.get();

getline → enter as a delimeter

② If your code not waiting for getline, use cin.get() to resolve this.

File → Input from file & print on consol.

while (!fin.eof()) → till end of the file

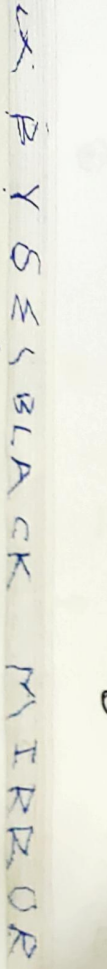
while (fin) → till data present in file

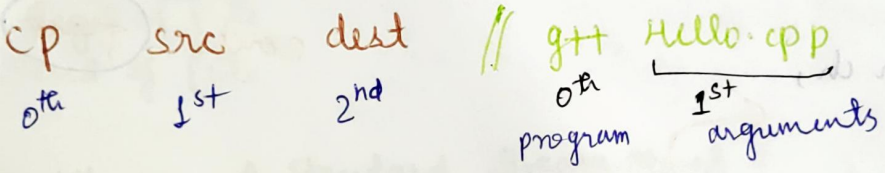
// Using char

```
fin >> ch;
cout << ch;
```

// Using string

```
getline (fin, str);
cout << str;
```






```
#include <iostream>
using namespace std;
int main (int argc, char * argv[], char *env[])
{
    cout << argc
    return 0; cout << argv[0] << argv[1] << argv[2];
    cout << env[0] << env[1] << env[2];
    return 0;
}
```

$ ./cmd Sachine Kambali

argc → 3
argv[0] → ./cmd
argv[1] → Sachine
argv[2] → Kambali

env[0] - Shell
env[1] -
env[2] -

} Environment variable

⊙ g++ work as file handling.

Take file → open file → read file → compile file → execute file

** When you want to write objects into file use character array instead of strings.

char array → cin.getline(name,50);

string → getline(cin, str);

⊙ For binary writing you need to use write function.

How much to write

From where to write

⊙ fout.write((char*)&s, sizeof(student));

address of student object

size of student object

Read

```
while (fin.read ((char *) & s, sizeof(student));
    {
      s.Display();
    }
    fin.close();
    return 0;
}
```

Code Reuse -

1. Function
2. Association
3. Inheritance
4. Template

Template

1. You can make generic container.
2. You can make generic functionalities.
3. Ease of access

- A function template uses no memory.
- No actual code is generated until the function named in the template is called.
-

Class template -

- Unlike functions, a class template is instantiated by suppling the type at object definition / declaration.

```
# include <iostream>
using namespace std;
template < class T>
class Joiner
{
```

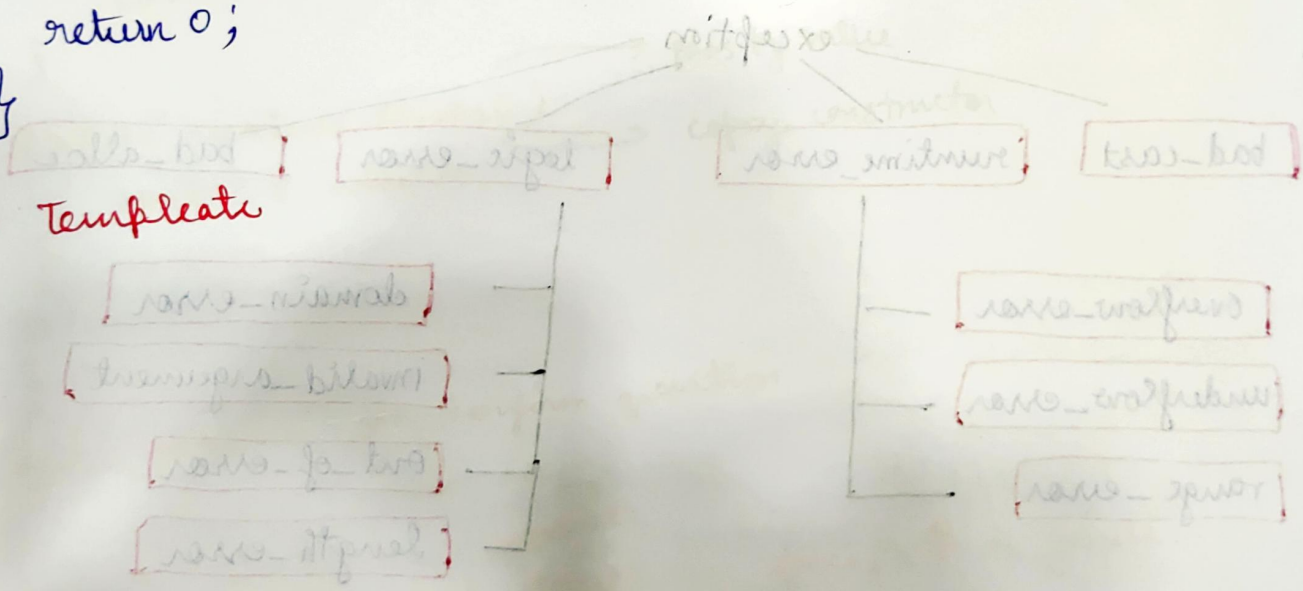
```
public:
    T combine (Tx, Ty){
      return x+y; }
    }
};
int main()
{
    Joiner <int>ij;
    Joiner <float> cj;
    cout << ij. combine (10,10);
    cout << cj. combine (10.10,20.20);
    return 0;
}
```

Why Queue

↓ Rate of processing / Rate of Production is low

Or decision making is low

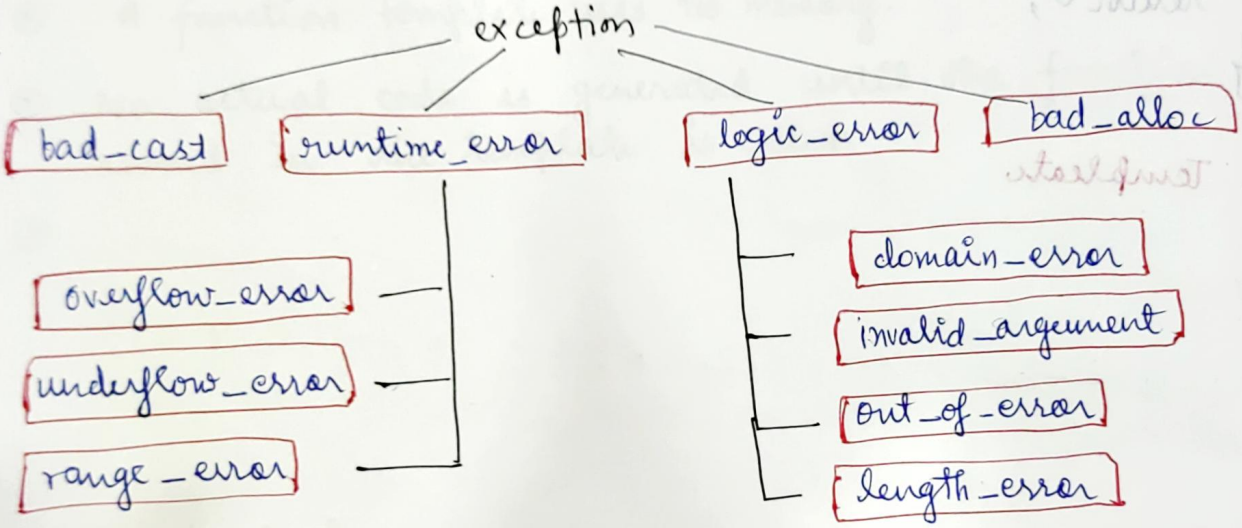
① Templeate



Exceptions -

- An exception is a abnormal condⁿ that occurs at run time & disturbs normal continuation of the program.
- When an exception occurs, the program must either terminate or jump to exception handling code.
- The special code for exception handling the is called an Exception Handler.

e.g. divide by zero, Array out of Bound etc.



Header files for using exceptions.

Exception class	Header file
exception	# include <exception>
runtime_error, logic_error & their subclasses	# include <stdexcept>
bad_alloc	# include <new>
bad_cast	# include <typeinfo>

⑥ when exception comes catch it & give another chance to try block.

try, catch & throw example.

```
try {
    float avg = Divide (10,0);
    cout << " Average = " << avg;
}
catch (runtime_error e)
    cout << e.what();
}
return 0;
}
```

→ pass by value

→ copy constructor

custom exception ** confirm question

Exception handler block should be - specific to generic sequence

RTTI and Advanced Type Cast -

- Upcasting atomatically done.
- While downcasting, you need to check i.e. type check & that type check is done by RTTI.

typeid() -

(int → i)

- Returns a object of type_info describing that type.
- typeid() can be used on any variable or type
-

```
template <class T>
bool isnumericType(T x)
{
    if (typeid(x) == typeid(short)) return true;
    if (typeid(x) == typeid(int)) return true;
    if (typeid(x) == typeid(double)) return true;
    if (typeid(x) == typeid(float)) return true;
    return false;
}
```

For user defined data type -

```
Class Student
typeid(Student).name();
```

→ 7 Student

Class name followed by class name character size

Advanced Type Casting -

<typeinfo>

- Implicit casting / Function call casting
- Explicit casting

1. cons_cast - for const to non-const & vice versa
2. reinterpret_cast - for pointer conversion
3. static_cast - data type conversion
4. dynamic_cast -

Volatile → No caching
threading

shared variable make volatile.

```
int main()
{
  const int a = 10;
  volatile int v = 20;
```

downcasting to happen means upcasting happened before